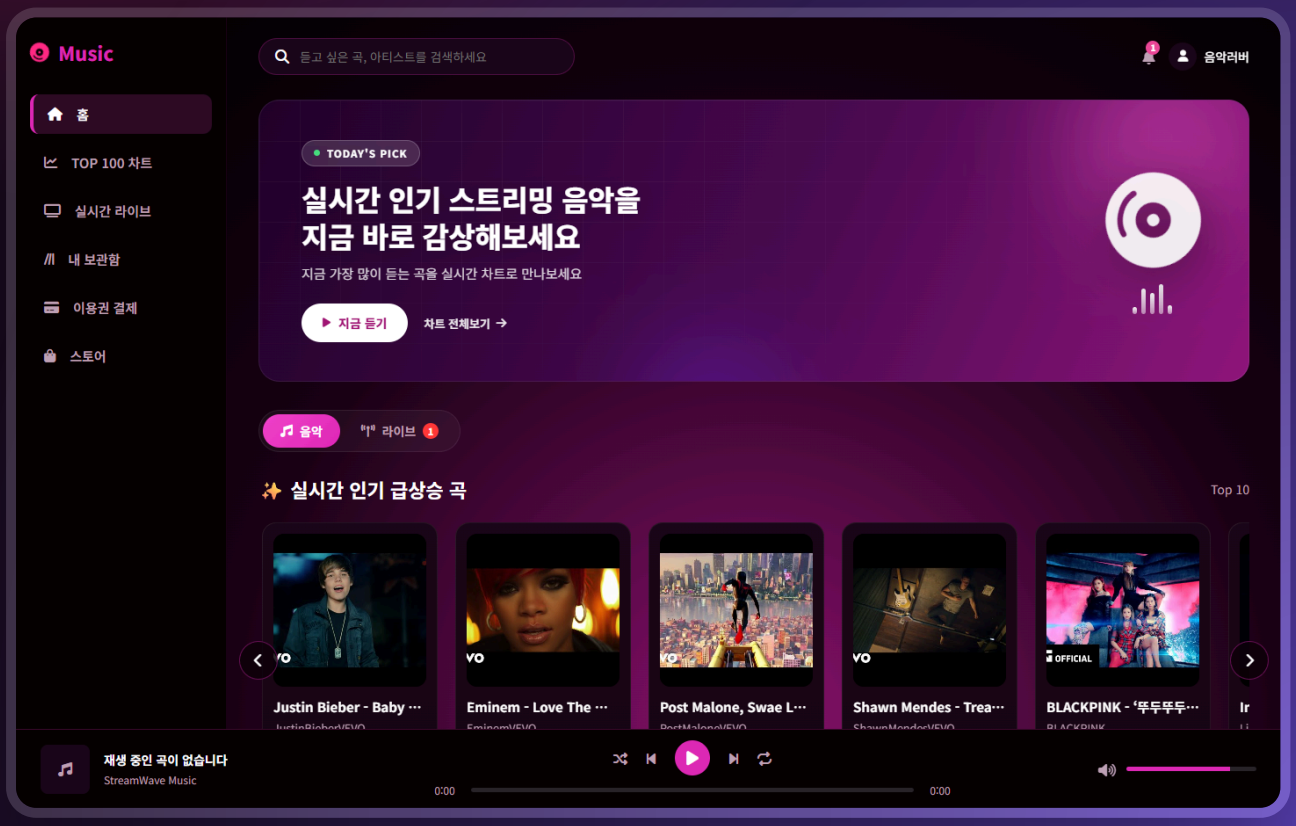
StreamWave

StreamWave

YouTube 음악 스트리밍과 라이브 방송을 한곳에서.
신청곡에 투표하고, 음표로 응원하고,
듣던 곡의 음반까지 바로 사는 음악 플랫폼

개발자	**천경호**
연락처	010-7757-5062
이메일	cjsrudgh98@gmail.com

구분	개인 프로젝트 · 풀스택 (기획 · 백엔드 · 프론트엔드 · 배포)
Backend	Java 25 · Spring Boot 4.1 · Spring Security · JPA · WebSocket(STOMP)
Frontend	React 18 · Vite 5 · hls.js · YouTube IFrame API
Infra	AWS EC2 · Docker Compose · Caddy · MySQL · Redis · SRS(RTMP/HLS)
배포	15-165-125-223.sslip.io
GitHub	github.com/cjsrudgh98-crypto/-MUSIC-main2



OVERVIEW

프로젝트 개요

음악은 혼자 듣기도 하지만, 누군가의 방송에서 함께 듣기도 합니다. 그런데 두 경험은 보통 서로 다른 서비스에 흩어져 있습니다. StreamWave는 **음악 스트리밍**과 **라이브 방송**을 하나로 묶어 "듣고 → 함께 듣고 → 응원하고 소장하는" 흐름을 한 곳에서 끝냅니다. 등급별 이용권 · 후원 · 스토어 결제와 환불, 채팅 관리, 관리자 기능까지 실제 운영을 전제로 만들었습니다.

80	14	39	3종
REST API 엔드포인트 (+ STOMP 3)	JPA 엔티티	백엔드 자동 테스트	결제 흐름 (이용권 · 음표 · 스토어)

01 LISTEN

듣기

YouTube 카탈로그 · 실시간 TOP 100 · 검색 · 플레이리스트 · 등급별 이용권

→

02 TOGETHER

함께 듣기

OBS → SRS 라이브 방송, 실시간 채팅, 신청곡 투표 → 1위 곡 자동 예약

→

03 SUPPORT

응원 · 소장

♪ 음표로 방송자 후원, 듣는 곡의 음반·굿즈를 스토어에서 바로 구매

🎵 음악 스트리밍

- 플레이리스트·채널·인기 차트 동기화, 비음악 영상 자동 정리
- 같은 곡 중복을 제목 + 재생시간(±5초)으로 묶어 대표 1곡
- TOP 100: 최근 24시간 → 누적 청취 → 조회수 순, 반복 재생은 시간당 1점
- 좋아요 · 최근 들은 곡 · 직접 만드는 플레이리스트
- 이용권 등급: 라이트 30곡 · 스탠다드(무제한 + 플레이리스트) · 프리미엄(+ 채팅 ♪ 배지 · 스토어 10% 할인)

📡 라이브 방송

- OBS RTMP → SRS → HLS, 방송 화면 자동 썸네일
- STOMP 채팅 · 신청곡 투표(1인 1표) · 채팅 관리(삭제·금지)
- OBS 채팅 오버레이 · 채팅 독, 다른 페이지에서도 미니 플레이어
- 팔로우 채널 방송 시작 알림

♪ 음표 · 스토어

- 음표(1음표=1원) 후원 → 채팅·OBS 화면에 음표 알림
- 재생 중인 곡의 관련 음반·굿즈를 플레이어 바에서 바로
- 토스페이먼츠 결제, 재고 원자적 차감, 취소·환불
- 관리자 상품 등록·곡 연결·주문 배송 상태 관리

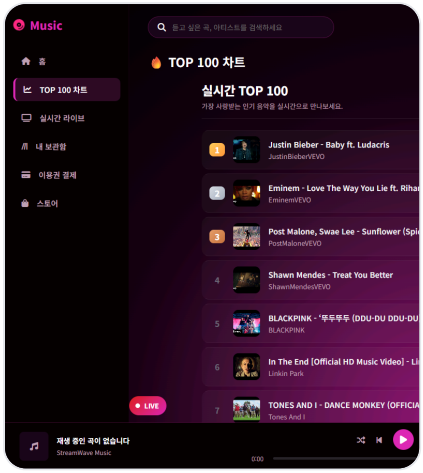
🔐 회원 · 보안

- 이메일 가입 + Kakao·Naver·Google 소셜 로그인
- 인증된 이메일만 계정 연결 (사전 가입 탈취 방지)
- 인증번호 5분 만료·5회 제한, 로그인 Rate Limit
- 권한 3단계 (회원 · 부 관리자 · 최고 관리자)

SCREENS

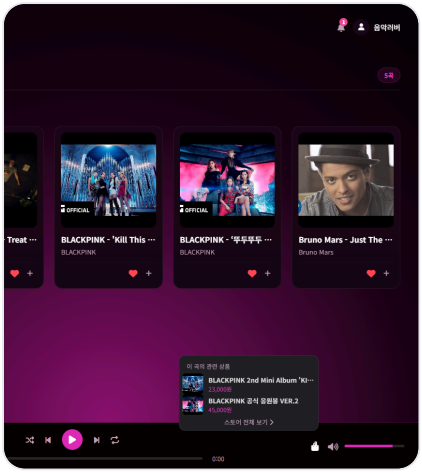
주요 화면 · 듣기와 구매

실제 서버를 로컬에서 띄워 촬영했습니다. 곡 카탈로그는 실제 데이터, 상품·방송·채팅은 촬영용 예시 데이터입니다.



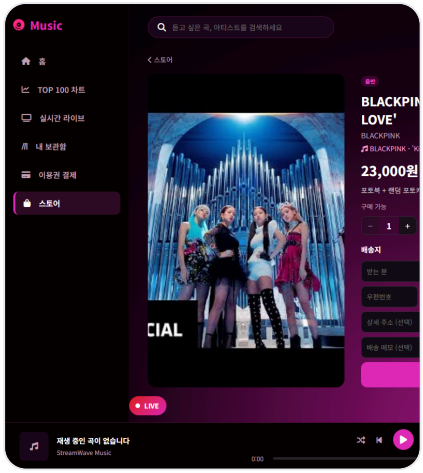
실시간 TOP 100

최근 24시간 청취가 먼저, 남는 자리는 서비스 누적 청취 → YouTube 조회수 순으로 채웁니다.



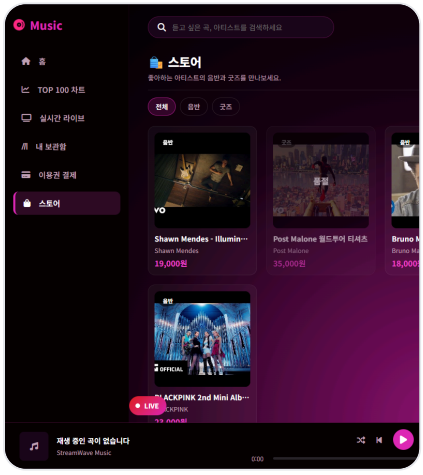
지금 듣는 곡의 관련 상품

곡에 연결된 음반, 같은 아티스트의 굿즈를 플레이어 바에서 바로 보여줍니다.



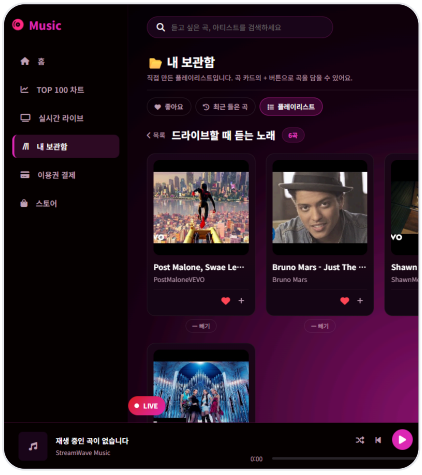
상품 상세 · 주문

수량·배송지를 입력하면 서버가 금액을 계산한 주문을 만들고 토스 결제로 이어집니다.



스토어

음반·굿즈 분류, 품절 표시. 상품은 삭제 대신 판매 중지로 내려 지난 주문 내역을 지킵니다.



내 플레이리스트

곡 카드의 + 버튼으로 담고, 전체 재생·이름 변경·빼기를 지원합니다.

스토어 주문 상태

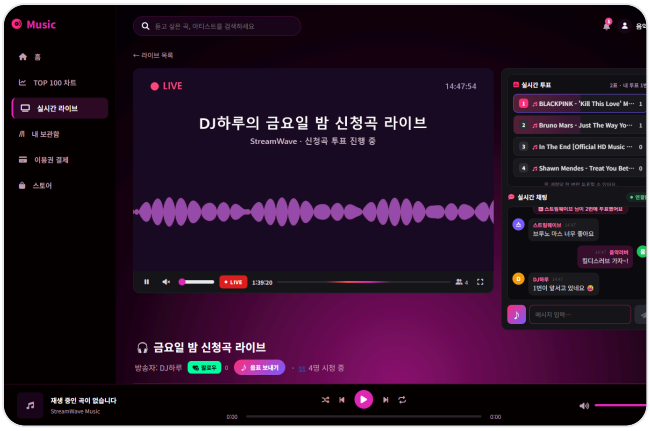
결제 대기 → 결제 완료 → 배송 중 → 배송 완료 · 취소·환불

구매자는 배송 전까지, 관리자는 배송 중까지 취소할 수 있고 결제 취소와 재고 복구가 함께 처리됩니다. 결제창에서 그만둔 주문은 하루 뒤 정리됩니다.

SCREENS

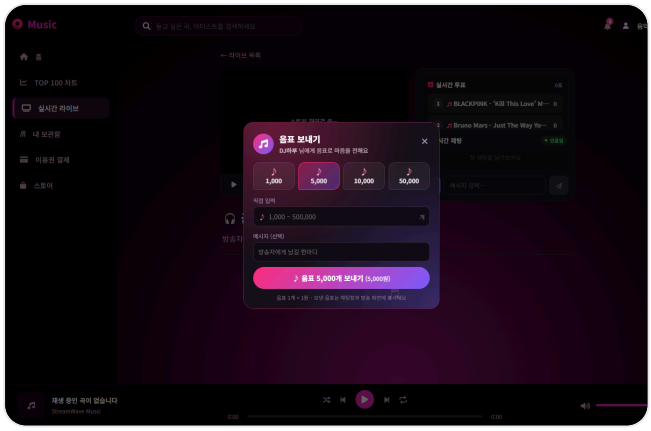
주요 화면 · 라이브와 관리자

라이브 화면의 영상은 ffmpeg 로 만든 테스트 방송입니다 (OBS 송출과 같은 HLS 경로로 재생).



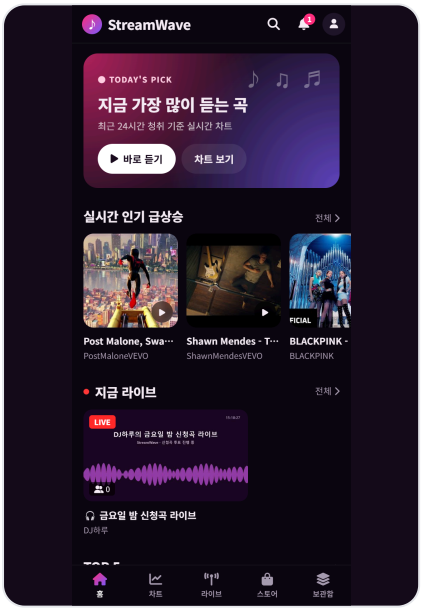
라이브 방송 · 신청곡 투표 · 채팅

시청자는 버튼이나 채팅 “투표1”로 1인 1표, 방송자가 “1위 곡 다음 재생”을 누르면 방송자 플레이어에 예약됩니다.



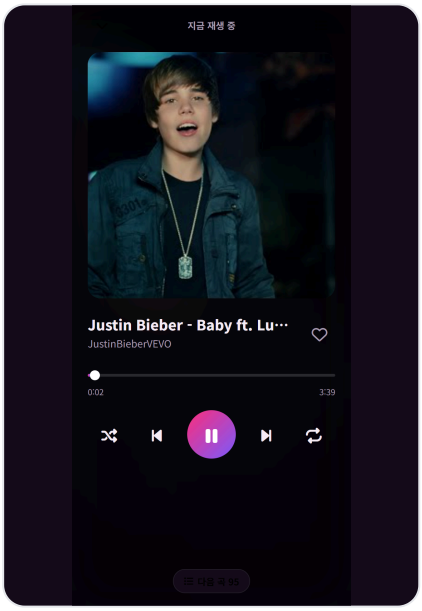
♪ 음표 보내기

1음표=1원. 결제가 승인되면 채팅창과 OBS 오버레이에 음표 모양 알림이 뜹니다.



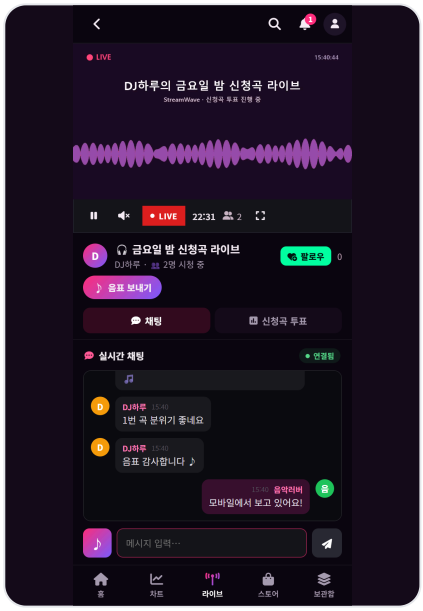
모바일 홈

데스크톱과 분리한 전용 화면. 하단 탭바 · 미니 플레이어.



전체화면 플레이어

미니 플레이어를 누르면 올라옴. 좋아요·관련 상품·다음 곡.



모바일 라이브

영상 아래 [채팅 | 신청곡 투표] 탭.

시연 영상 (2분 29초)

검색·재생 → 관련 상품 → 주문·토스 결제창 → TOP 100 → 라이브 입장 → 투표·채팅 → 음표 → 플레이리스트 → 이용권 등급 → 관리자 스토어 관리 → 모바일 전용 화면

`github.com/cjsrudgh98-crypto/-MUSIC-main2/blob/main/docs/demo/StreamWave-demo.mp4`

ARCHITECTURE

시스템 구조

AWS EC2 한 대에서 Docker Compose 로 MySQL · Redis · SRS · Spring · Caddy 를 띄웁니다. Caddy 가 HTTPS 를 맡고 `/api` · `/ws` 는 Spring 으로, HLS 파일만 SRS 로 보냅니다.

Client

React 18 SPA
Vite · React Router 6
데스크톱 / 모바일 전용 화면 분리

플레이어
YouTube IFrame API
라이브 hls.js

실시간
STOMP over SockJS
채팅 · 투표 · 알림

OBS Studio
RTMP 송출
채팅 오버레이 · 독

Spring Boot 4.1 (Docker)

Security
Bearer 토큰(Redis) + 세션 · OAuth2 (Kakao/Naver/Google) · Rate Limit · 오류 ID 응답

Controller
음악 · 차트 · 라이브 · 채팅(STOMP) · 음표 · 스토어 · 결제 · 플레이리스트 · 관리자 — REST 80 + STOMP 3

Service
YouTubeApi(다중 키) · Chart(24h 버킷) · SongVote · ChatModeration · Donation · Shop · Payment · Notification

Data
MySQL (JPA 엔티티 14) · Redis (토큰 · 차트 · 투표 · 시청자 수 · 알림 · 채팅 금지)

Scheduler
YouTube 동기화 · 이용권 만료 알림 · 결제 대기 정리 · 스냅샷

SRS 연동
on_publish 훅에서 송출 키 검증 → 방송 ON/OFF · 투표 정리

External

YouTube Data API v3
playlistItems · videos · search (할당량 관리)

토스페이먼츠
결제위젯 · 승인 · 취소

SRS 6
RTMP → HLS · 썸네일

OAuth2
Kakao · Naver · Google

AWS
S3 + CloudFront · Gmail SMTP

기술 스택

Backend	Java 25, Spring Boot 4.1 (Web, Data JPA, Security, OAuth2 Client, WebSocket/STOMP, Mail, Validation), springdoc-openapi
Frontend	React 18, Vite 5, React Router 6, Axios, @stomp/stompjs + SockJS, hls.js, react-youtube
Data	MySQL 8, Redis
Live	SRS 6 (RTMP → HLS, HTTP 훅), ffmpeg 썸네일 캡처, OBS
Test	JUnit 5, Spring Boot Test, Mockito (`@MockitoBean`)
Infra	AWS EC2 (탄력적 IP), Docker Compose (MySQL · Redis · SRS · Spring · Caddy), Caddy 자동 HTTPS

PROBLEM SOLVING

기술적 도전과 해결

돈과 실시간 방송이 오가는 서비스라, 기능을 “동작하게” 만든 뒤에 악용과 장애 상황을 먼저 찾아 막았습니다.

01 라이브 방송 가로채기 방지

문제 스트림 키가 곧 HLS 파일 이름이라, 시청자가 주소에서 키를 보고 같은 키로 송출해 남의 방송을 가로챌 수 있었음. SRS 웹훅도 외부에서 호출 가능

해결 공개 `playbackId` 와 비밀 `streamKey` 를 분리 (OBS 키 = `{playbackId}?key={streamKey}`). `on_publish` 훅에서 키를 상수 시간 비교로 검증해 틀리면 송출 거부, 웹훅 경로는 Caddy 에서 외부 차단

결과 시청자에게 보이는 주소로는 송출 불가 · 외부에서 방송 강제 종료 불가

02 결제 금액 위변조 · 재고 정합성

문제 이용권·음표·스토어 모두 클라이언트가 보낸 금액을 믿으면 조작 가능. 동시에 결제하면 재고보다 많이 팔릴 수 있음

해결 결제 전에 서버가 금액을 고정한 PENDING 주문 생성 → 승인 요청 금액이 다르면 토스를 부르지 않음. `stock = stock - :qty WHERE stock >= :qty` 로 원자적 차감 후 승인, 실패 시 트랜잭션 롤백

결과 금액 조작·중복 승인·초과 판매 차단, 환불 시 재고 복구 — 테스트로 검증

03 실시간 채팅의 신원과 1인 1표

문제 클라이언트가 보낸 발신자 이름을 그대로 써서, 방송자 닉네임으로 투표창 명령을 보내거나 이름을 바꿔 무제한 투표 가능

해결 웹소켓 핸드셰이크에서 인증된 STOMP `Principal` 로 발신자를 서버가 결정. 투표는 사용자 PK 기준 Redis `SETNX` , 음표 알림 같은 시스템 메시지 타입은 서버만 발송. 연결당 도배 제한

결과 사칭·중복 투표 차단 · 방송자 채팅 관리(삭제·채팅 금지) 기반 마련

04 YouTube API 하루 할당량

문제 `search.list` 는 호출당 100유닛(하루 약 100회). 비로그인 사용자가 검색어를 바꿔가며 할당량을 소진할 수 있었음

해결 동기화는 1유닛짜리 `playlistItems` · `videos` 위주로. 키 여러 개를 두고 **할당량 사유의 403/429**일 때만 봉인 후 다음 키로. 검색 보강은 로그인 사용자만, 사이트 전체 시간당 10회·하루 40회 상한

결과 검색 남용으로 동기화·관리자 등록이 막히는 상황 방지

05 “실시간” 차트를 진짜 실시간으로

문제 Redis 점수가 줄지 않는 누적값이라 한번 오른 곡이 계속 상위에 남았고, 같은 곡 반복 재생으로 순위 조작 가능

해결 시간대별 버킷(`chart:h:{epochHour}`)에 쌓고 조회 시 최근 24개를 합산(30초 캐시). 같은 사용자·곡은 한 시간에 1점

결과 최근 24시간 청취가 그대로 반영되는 차트 · 반복 재생 조작 차단. 청취가 뜸한 날 차트가 조회수 순으로 통째로 바뀌던 문제는 누적 청취 순 단계를 넣어 해결

06 소셜 로그인 계정 탈취

문제 이메일로 기존 계정에 연결하는데 카카오는 미인증 이메일을 줄 수 있음. 회원가입에 이메일 인증이 없어 남이 먼저 내 이메일로 가입해 둘 수도 있음

해결 제공자가 인증한 이메일만 허용(`is_email_verified` , `email_verified`). 같은 이메일의 일반 계정에 처음 연결될 때 기존 비밀번호 무효화. 토큰은 URL 쿼리 대신 프래그먼트(#)로 전달

결과 미인증 이메일·사전 가입을 통한 계정 탈취 차단 — 회귀 테스트로 고정

07 곡 삭제 FK 오류로 목록 전체가 실패

문제 비음악 영상을 목록 조회 중에 정리하는데, 좋아요·청취기록이 걸린 곡을 지우면 커밋 시점에 FK 오류 → 홈 화면 곡 목록 요청 전체가 500

해결 재현 테스트로 원인 확인 후 모든 삭제 경로를 `MusicRemover` 로 통일(참조 기록 먼저 정리). 새 테이블은 `ON DELETE CASCADE / SET NULL` 로 설계

결과 정리 대상 곡이 있어도 목록 조회 정상 · 플레이리스트·상품 연결도 자동 정리

08 라이브 화면 레이아웃 · 모바일 전용 화면

문제 페이지 래퍼에 폭이 없어 내용 폭(약 760px)으로 줄어들어 영상 400px, 채팅은 메시지 한 줄. 새 채팅마다 페이지 전체가 스크롤. 휴대폰에서는 고정 사이드바 때문에 영상이 78px

해결 래퍼 폭 지정 · 채팅 최소 높이 · 투표창 45% 상한, 채팅 목록만 스크롤. 768px 이하는 화면 틀을 분리해 하단 탭바 · 미니/전체화면 플레이어 · 전용 페이지로 (데이터·플레이어·채팅 부품은 공유)

결과 1440px 기준 영상 400px → 772px · 390px 휴대폰에서 가로 넘침 없음 (브라우저로 실측)

QUALITY

테스트 · 검증

결제·권한·실시간 기능은 실제 MySQL·Redis 를 붙인 통합 테스트로, 외부 결제는 Mockito 로 대체해 검증합니다. 화면은 실제 브라우저로 띄워 측정·촬영했습니다.

결제 · 주문	음표 주문·승인 금액 위변조 거절, 중복 승인 1회 처리 · 스토어 재고 차감·환불·상태 전이 · 이용권 만료 알림 대상 선정
이용권	등급별 기능 판정(관리자 전체) · 라이트 30곡 차감과 같은 곡 재차감 없음 · 같은 등급 연장 예약 / 다른 등급 즉시 적용 · 예전 이용권 등급 대응 · 프리미엄 스토어 할인 · 채팅 배지 위조 방지
권한 · 보안	남의 주문·플레이리스트 접근 거부 · 소셜 계정 연결 규칙(비밀번호 무효화, 닉네임 중복) · 카카오 미인증 이메일 · Rate Limit
실시간	투표 1인 1표 · 투표 옵션과 카탈로그 곡 연결 · 채팅 금지 TTL · 차트 시간당 1점
회귀	좋아요가 걸린 정리 대상 곡이 있어도 곡 목록 조회 성공 (실제 재현했던 FK 오류)
화면	Playwright + Edge 로 1440 / 1024 / 768 / 390px 에서 레이아웃 실측, 시연 영상·스크린샷 자동 생성 (`docs/demo`)

NEXT

앞으로 개선할 점

운영 · 품질

- GitHub Actions CI (테스트 · 빌드 · Docker 이미지)
- 프론트 단위 테스트 · E2E 테스트를 CI 에 포함
- `ddl-auto: update` → Flyway 마이그레이션
- Rate Limit · 차트 캐시를 Redis 기반으로 옮겨 수평 확장 대비

기능

- 방송자 음표 정산(출금) · 택배사 배송 조회 연동
- 상품 이미지 S3 업로드를 관리 화면에 연결
- 실시간 알림을 사용자별 STOMP 큐로 전환
- 홈 화면 카탈로그 조회 캐싱

배포 사이트 · 저장소 · 시연 영상

`https://15-165-125-223.sslip.io` (관리자 체험 계정은 README 참고)

`github.com/cjsrudgh98-crypto/-MUSIC-main2`

README 에 실행 방법(로컬 / AWS EC2 + Docker Compose), 화면, 시연 영상이 있습니다.